

Machine Learning

2.1 Qué datos se usan para entrenar

El modelo se entrena con `dataset_limpio.csv` y usando **título + texto combinados** como entrada (`titulo_limpio + ' ' + texto_limpio`).

- Con TF-IDF, el oversampling del dataset balanceado duplica textos completos, generando vectores idénticos repetidos que no aportan información nueva al modelo y pueden inflar métricas si un duplicado cae en train y otro en test.
- La API en producción envía título y texto juntos; entrenar solo con texto crea una diferencia entre lo que el modelo aprendió y lo que recibe en producción (*train/serve skew*), lo que baja el rendimiento real sin que sea un problema del algoritmo.

2.2 Vectorización TF-IDF

Se subió `max_features` de 5,000 a 60,000 (con `min_df=5` y bigramas). Con 38 mil documentos técnicos, 5,000 términos era vocabulario insuficiente y dejaba fuera variantes técnicas válidas (nombres de librerías, comandos, etc.). El cambio, medido directamente, subió el F1 macro de 0.70 a 0.75 solo por esto (junto con el cambio de título+texto).

2.3 Línea base (baseline)

Se agregó un `DummyClassifier` que siempre predice la categoría más frecuente, como punto de referencia mínimo. Sin un baseline no hay forma de saber si un F1 de 0.76 es bueno o malo. El baseline dio F1 macro ≈ 0.03 , así que el modelo final resultó ser **24.5 veces mejor que el azar** es la cifra que sostiene que el modelo aprendió algo real, no solo aprovechó el desbalance de clases.

2.4 Entrenamiento y comparación de algoritmos

Se entrenaron y compararon 4 modelos (Regresión Logística base, Naive Bayes base, y ambos ajustados por hiperparámetros), más el baseline, en una sola tabla comparativa. La rúbrica pedía explícitamente evaluar Naive Bayes como alternativa y comparar algoritmos, no solo entrenar uno.

2.5 Métricas: por qué macro y no solo weighted

Se calculan Accuracy, Precision, Recall y F1 tanto en versión `macro` como `weighted`, pero la selección del mejor modelo se basa en **F1 macro**. El dataset tiene categorías desbalanceadas (Seguridad con 2,792 registros vs. Frontend con 5,402, ratio $\sim 1.9x$). `F1 weighted` pondera por tamaño de clase y puede disimular que el modelo falla en la categoría chica mientras acierta en las grandes. `F1 macro` trata a todas las categorías por igual, así que es el criterio más honesto para saber si el modelo realmente aprendió a distinguir Seguridad y no solo las categorías grandes.

2.6 Ajuste de hiperparámetros sin fuga de datos

El `GridSearchCV` (y la validación cruzada final) se ejecutan sobre un **Pipeline completo** (`TfidfVectorizer` + clasificador), no sobre una matriz TF-IDF ya vectorizada de antemano. Si el TF-IDF se ajusta una sola vez antes de partir en folds de validación cruzada, cada fold ya "vio" información del resto del conjunto de entrenamiento a través del vocabulario e IDF global, lo que infla artificialmente el resultado de la validación cruzada. Ajustando el TF-IDF dentro de cada fold del Pipeline, la validación cruzada refleja el desempeño real esperado con datos nuevos.

2.7 Selección del mejor modelo y validación

Se seleccionó el modelo con mayor F1 macro entre los 5 candidatos (Regresión Logística ajustada, $C=4.0$, F1 macro = 0.759), y se confirmó con validación cruzada de 5 folds (F1 macro = 0.752 ± 0.006). La cercanía entre el resultado en test (0.759) y en validación cruzada (0.752) confirma que el desempeño es consistente y no un resultado afortunado de un único split de datos.

2.8 Interpretabilidad: términos más decisivos por categoría

Se extrajeron, de los coeficientes (`.coef_`) de la Regresión Logística, las palabras con más peso por categoría (ej. Frontend: css, html, react — Seguridad: token, encryption — DevOps: docker, aws). Es la evidencia de que el modelo aprendió vocabulario técnico real y coherente por categoría, y no relaciones espurias o ruido.

2.9 Formato de entrega del modelo

El modelo final se guarda como **un solo objeto** (`Pipeline` con los pasos `"tfidf"` y `"clf"`) en `modelo_techmind_v2.joblib`, en vez de guardar el vectorizador y el clasificador en archivos separados. El backend del proyecto espera un objeto único que soporte `modelo.predict_proba(texto_crudo)` y `modelo.named_steps["tfidf"]`. Con dos archivos sueltos, el backend no podía cargar el modelo correctamente.